

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Проектирование и реализация баз данных

Лабораторная работа № 2-1

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлуниин Александр Александрович

г. Санкт-Петербург 2026 г

Содержание

Содержание.....	2
1. Предметная область и источники данных.....	2
Анализ структуры данных:.....	2
2. Проектирование схемы NoSQL (MongoDB).....	3
Структура коллекции teams (JSON):.....	3
Структура коллекции games (JSON):.....	3
3. Обоснование проекта.....	6

1. Предметная область и источники данных

Предметная область: Информационная система “База данных хоккейных матчей и команд NHL”.

Источник данных: Открытое API Национальной хоккейной лиги (NHL Stats API).

Целью работы является создание базы данных для хранения спортивной статистики, информации о хоккейных франшизах и детальной хронологии матчей.

Анализ структуры данных и эндпоинтов API:

Сбор данных будет осуществляться через основные эндпоинты NHL API. В процессе работы (на этапе ETL-преобразования) сырые данные из API нормализуются и переименовываются для удобного хранения в БД.

Выделены следующие ключевые сущности:

1. Team (Команда): статические данные о клубах.
 - Эндпоинт: /api/v1/teams

- Анализ полей: API возвращает сложную структуру (id, name, venue.city, venue.name, division.name, conference.name, firstYearOfPlay). При загрузке в БД эти поля сглаживаются (flattening) и переименовываются в более лаконичные ключи (city, arena, division, conference, founded_year). Данные атрибуты ожидаются в типовом ответе API и используются как обязательные в проектируемой схеме.

2. Game (Матч): динамические данные о событии.

- Эндпоинты: /api/v1/schedule и /api/v1/game/{gamePk}/feed/live.
- Анализ полей: из ответов извлекаются метаданные (gamePk, gameDate, status.abstractGameState), ссылки на участников (teams.away.team.id, teams.home.team.id), а также итоговый счет (linescore.teams) и командная статистика (boxscore.teams).

3. Game Event (Событие матча - Гол): хронология заброшенных шайб.

- Эндпоинт: вложенный массив liveData.plays.scoringPlays.
- Анализ полей: извлекаются период, время, ID команды и авторы. Массив ассистентов (assists) является опциональным. Сущность не выносится в отдельную коллекцию, а вкладывается в документ матча.

2. Проектирование схемы NoSQL (MongoDB)

Схема базы данных состоит из двух коллекций. В качестве первичных ключей (_id) используются уникальные целочисленные идентификаторы из NHL API (team_id и gamePk).

- Основная коллекция - games (информация о матчах).

- Вспомогательная коллекция - teams (справочник клубов).

2.1. Структура коллекции teams (Справочник)

Пример документа (JSON):

```
{  
  "_id": 14,  
  "name": "Tampa Bay Lightning",  
  "city": "Tampa",  
  "arena": "Amalie Arena",  
  "conference": "Eastern",  
  "division": "Atlantic",  
  "founded_year": 1992  
}
```

Спецификация типов данных:

- _id: Integer (Уникальный идентификатор команды из API)
- name, city, arena, conference, division: String (Используются как обязательные)
- founded_year: Integer

2.2. Структура коллекции games (Основная)

Пример документа (JSON):

```
{  
  "_id": 2023020123,  
  "date": "2023-11-15T19:00:00Z",
```

```
"status": "Final",

"home_team_id": 14,

"away_team_id": 8,

"score": {

  "home": 4,

  "away": 2

},

"team_stats": {

  "home_shots": 34,

  "away_shots": 28,

  "home_pim_minutes": 10,

  "away_pim_minutes": 8

},

"goals": [

  {

    "period": 1,

    "time": "12:34",

    "team_id": 14,

    "scorer_name": "Nikita Kucherov",

    "assists": ["Steven Stamkos", "Victor Hedman"]

  }

]
```

}

Спецификация типов данных:

- `_id`: Integer (Уникальный идентификатор матча gamePk из API)
- `date`: ISODate (в MongoDB) / ISO 8601 строка (в представлении JSON)
- `status`: String (Допустимые значения: "Preview", "Live", "Final")
- `home_team_id`, `away_team_id`: Integer (Ссылки на `_id` коллекции teams)
- `score`, `team_stats`: Object (вложенные объекты с Integer значениями)
- `goals`: Array of Objects (Массив вложенных документов. Поле `assists` - Array of Strings, может быть пустым).

3. Обоснование проекта

В схеме применен гибридный подход проектирования: сочетание нормализованных справочников и денормализованных игровых событий.

Использование вложенных документов (Embedded Documents): Статистика и массив голов (`score`, `team_stats`, `goals`) вложены в документ матча.

- Альтернативный вариант: Изначально рассматривался вариант вынесения сущности Game Event в отдельную коллекцию `game_events`. Однако от него решено было отказаться. Главный паттерн доступа к спортивной БД - запрос карточки матча целиком (кто играл, какой счет, кто забил). При отдельных коллекциях потребовалось бы выполнять ресурсоемкую операцию \$lookup для каждого матча.
- Безопасность вложения: В хоккее объем массива событий в пределах одного матча остается небольшим (обычно не более

10-15 записей). Даже с учетом овертаймов этот массив гарантированно не превысит лимит документа MongoDB в 16 МБ, что делает паттерн embedding абсолютно безопасным и максимально производительным для чтения.

Использование ссылок (References) и нативных ID: Связь с командами реализована через целочисленные `home_team_id` и `away_team_id`. Информация о самих клубах выделена в отдельную коллекцию `teams`. За сезон команды играют тысячи матчей, и вложение данных о стадионе или дивизионе в каждый матч привело бы к колоссальной избыточности и проблемам при обновлении данных (например, при смене названия арены). Использование естественных ID из API вместо сгенерированных `ObjectId` позволяет связывать документы напрямую, без дополнительных запросов к БД на этапе парсинга.

Вывод

В результате этапа 2-1 была спроектирована схема NoSQL-базы данных для хранения информации о хоккейных командах и матчах NHL. В ходе работы были выделены основные сущности предметной области, определены их атрибуты и выбраны подходы к хранению данных в MongoDB. Полученная схема учитывает структуру исходных JSON-данных API и ориентирована на эффективное выполнение типовых запросов к спортивной статистике